



Diseño Estructurado y Normalización de Bases de Datos

Bienvenido a la Unidad 4, donde exploraremos metodologías de diseño de bases de datos profesionales y eficientes. Aprenderemos enfoques de diseño estructurado y técnicas de normalización que ayudan a crear sistemas de bases de datos robustos y fáciles de mantener.

Agenda del curso

1

Fundamentos del diseño de bases de datos

Importancia de un diseño y metodologías adecuados

2

Fases de diseño

Etapas de diseño conceptual, lógico y físico

3

Proceso de normalización

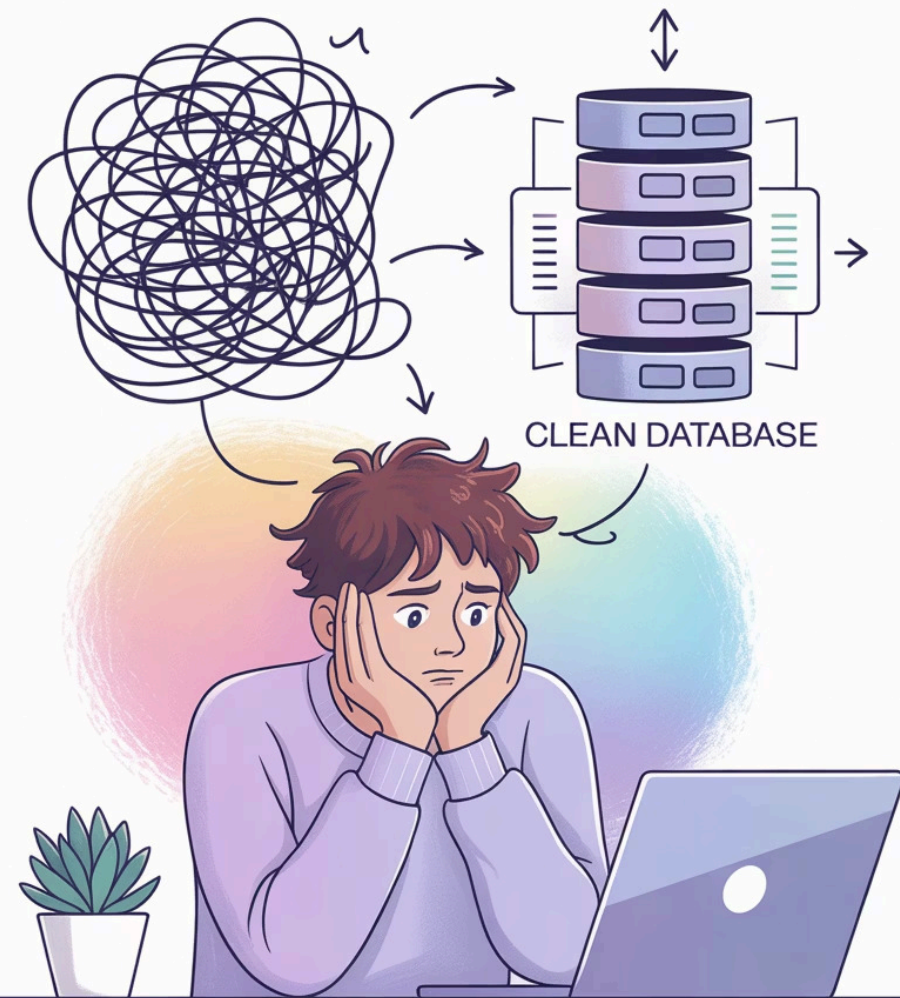
Primera, segunda y tercera formas normales

4

Aplicaciones prácticas

Ejemplos y ejercicios del mundo real

Avanzaremos desde la comprensión de por qué es importante un diseño adecuado hasta la implementación de técnicas de normalización en escenarios prácticos.



¿Por qué es fundamental un buen diseño de bases de datos?

Prevención de errores

Las bases de datos bien diseñadas minimizan las anomalías e inconsistencias de los datos que pueden provocar graves problemas en el sistema.

Optimización del rendimiento

Las bases de datos correctamente estructuradas mejoran significativamente la velocidad de búsqueda y el rendimiento general del sistema.

Escalabilidad

Una base de datos sólida facilita la expansión del sistema a medida que crecen o cambian los requisitos empresariales.

Al igual que la construcción de una casa, comenzar con la base de datos correcta evita problemas costosos más adelante.

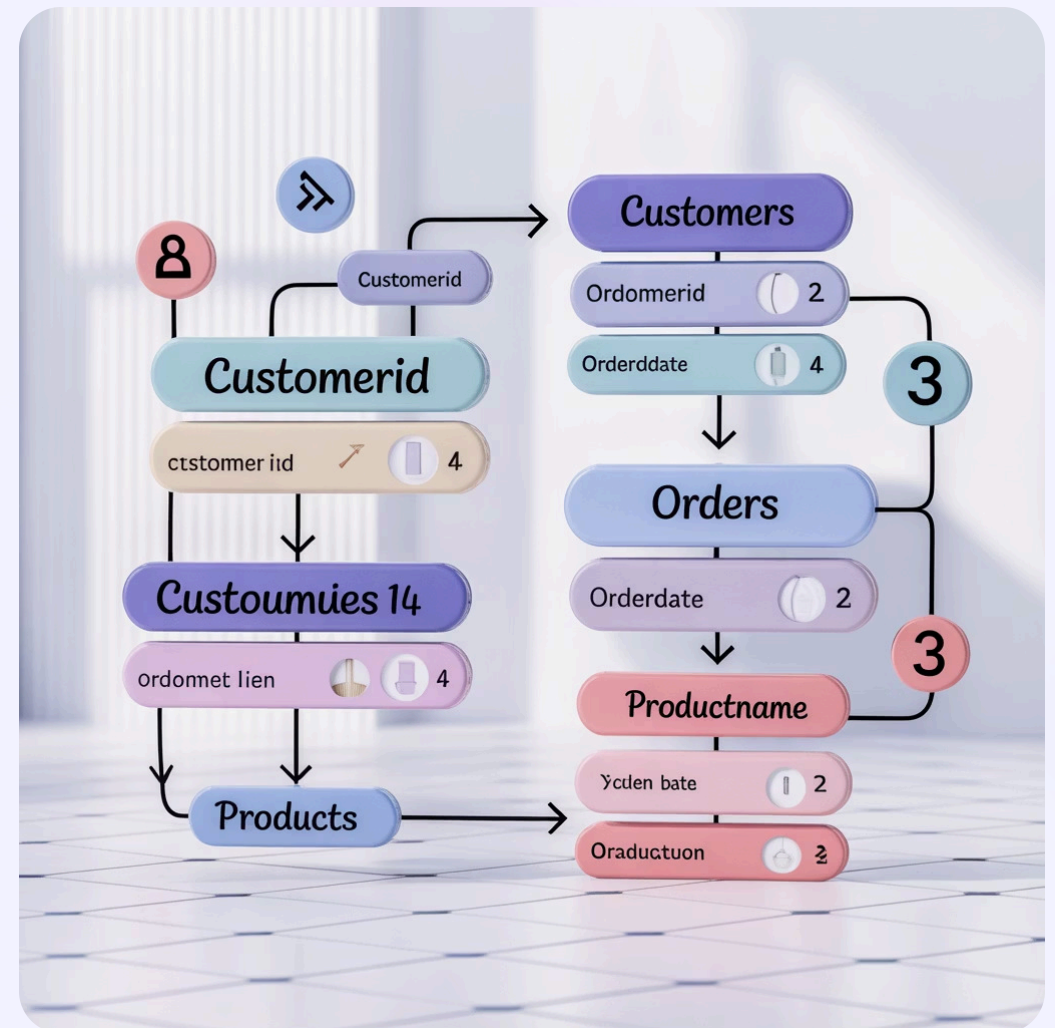
Metodologías de diseño

Modelo Entidad-Relación

El modelo E-R sigue siendo el estándar de la industria para el diseño conceptual de bases de datos. Proporciona una representación visual de:

- Entidades (objetos o conceptos)
- Relaciones entre entidades
- Atributos que describen las entidades

Este enfoque tiende un puente entre los requisitos empresariales y la implementación técnica.



Otros enfoques estructurados

- Diagramas UML
- Narrativas de requisitos
- Mapeo de historias de usuario
- Diseño basado en el dominio

Fases del diseño de bases de datos

Diseño conceptual

Se centra en **qué** datos deben almacenarse

- Identificar entidades y relaciones
- Definir reglas de negocio de alto nivel
- Crear diagramas E-R

Diseño lógico

Se centra en **cómo** se representan los datos en el modelo relacional

- Transformar el modelo conceptual en tablas
- Definir claves primarias y externas
- Aplicar reglas de normalización

Diseño físico

Se centra en los detalles de **implementación**

- Elegir un SGBD específico
- Definir estructuras de almacenamiento e índices
- Implementar medidas de seguridad

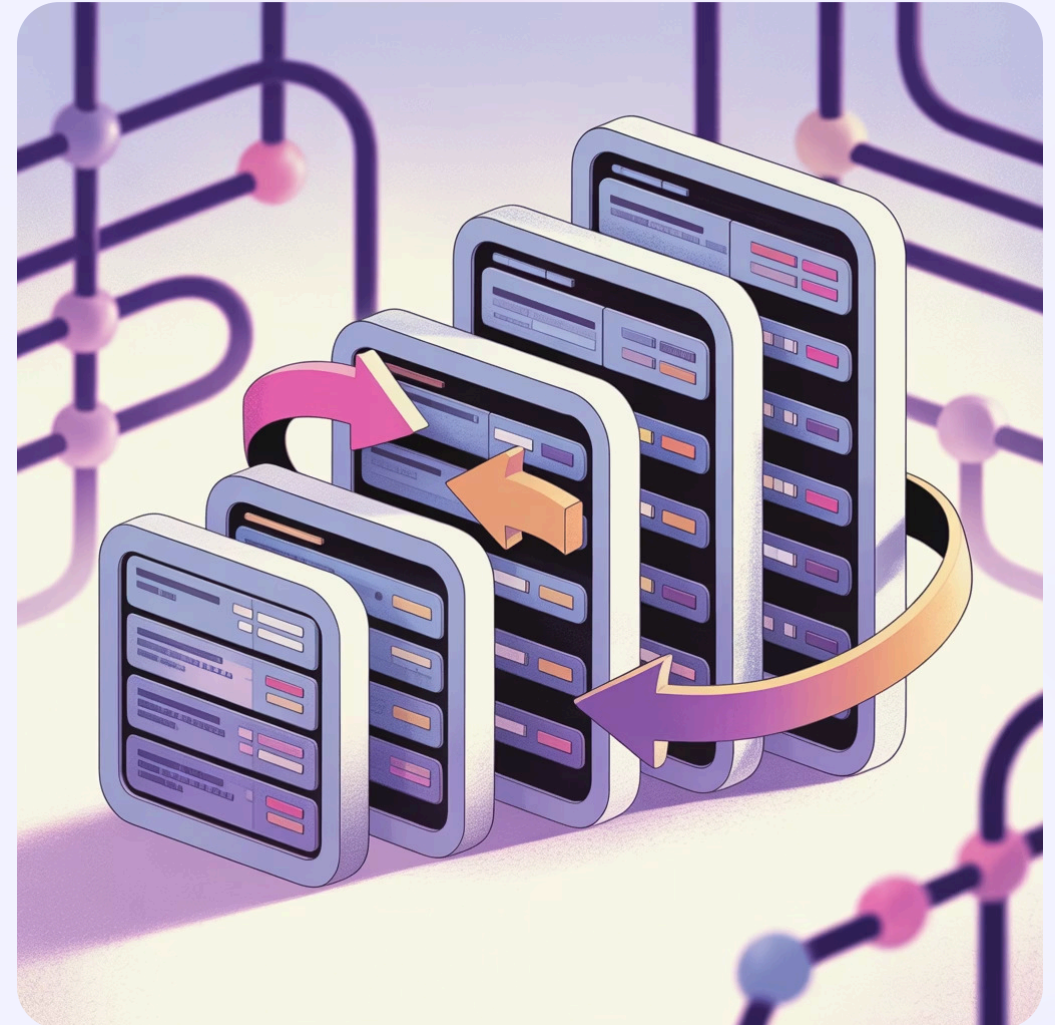
Cada fase se basa en la anterior, creando un sistema de base de datos completo y coherente.

Introducción a la normalización

La normalización es un proceso sistemático para organizar los datos en una base de datos relacional para:

- Eliminar la redundancia y las dependencias inconsistentes
- Reducir las anomalías de los datos (inserción, actualización, eliminación)
- Mejorar la integridad y la eficiencia de los datos

El proceso implica la aplicación de una serie de reglas llamadas **formas normales**, cada una basada en la anterior.



La mayoría de los diseños de bases de datos pretenden alcanzar al menos la Tercera Forma Normal (3FN), aunque existen formas normales superiores para necesidades específicas.

La necesidad de la normalización

Problemas con los datos no normalizados

- La redundancia de datos desperdicia espacio de almacenamiento
- Las anomalías de actualización crean inconsistencias
- Las anomalías de inserción impiden el almacenamiento de ciertos datos
- Las anomalías de eliminación causan la pérdida involuntaria de datos

Beneficios de los datos normalizados

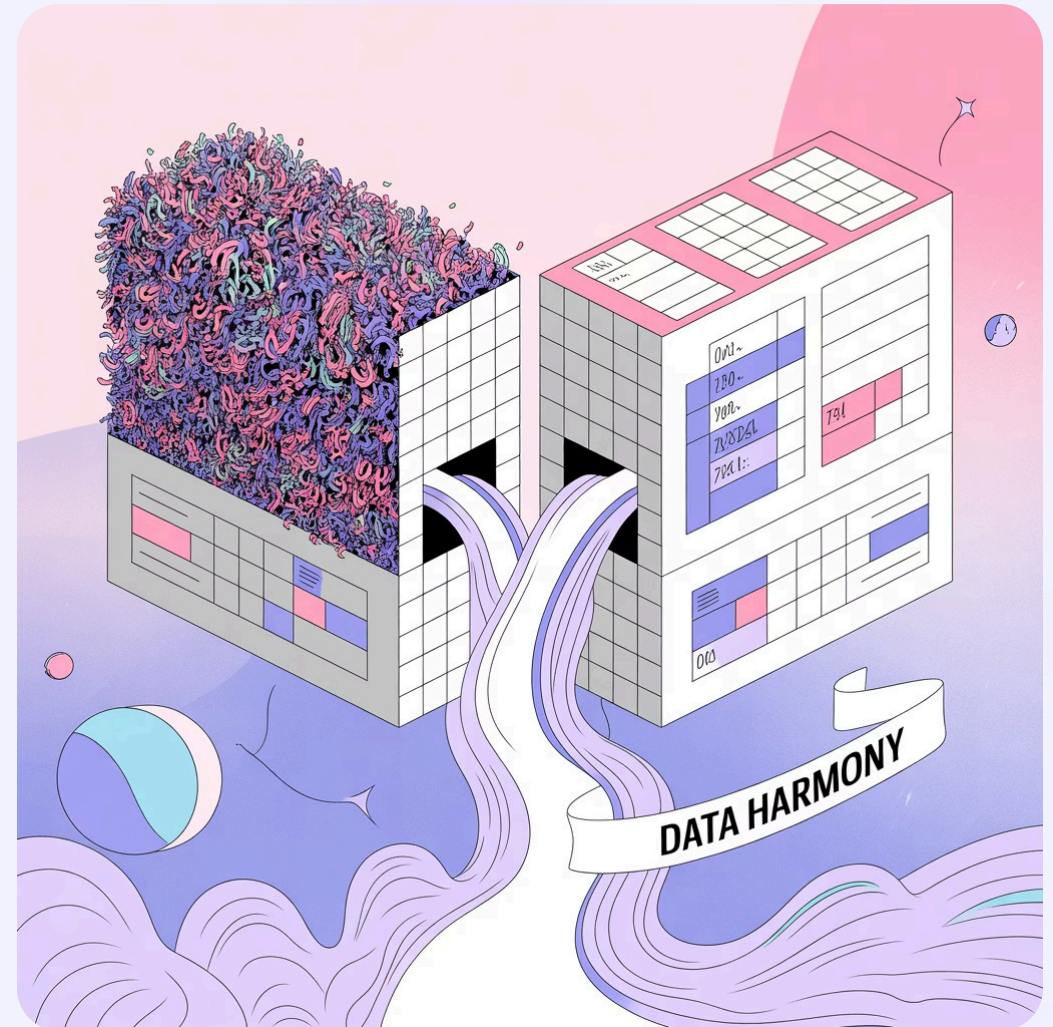
- Los datos se almacenan de manera eficiente con una redundancia mínima
- La estructura de la base de datos es más flexible y adaptable
- La integridad de los datos se conserva mejor
- Las consultas pueden ser más intuitivas y eficientes



Primera Forma Normal (1FN)

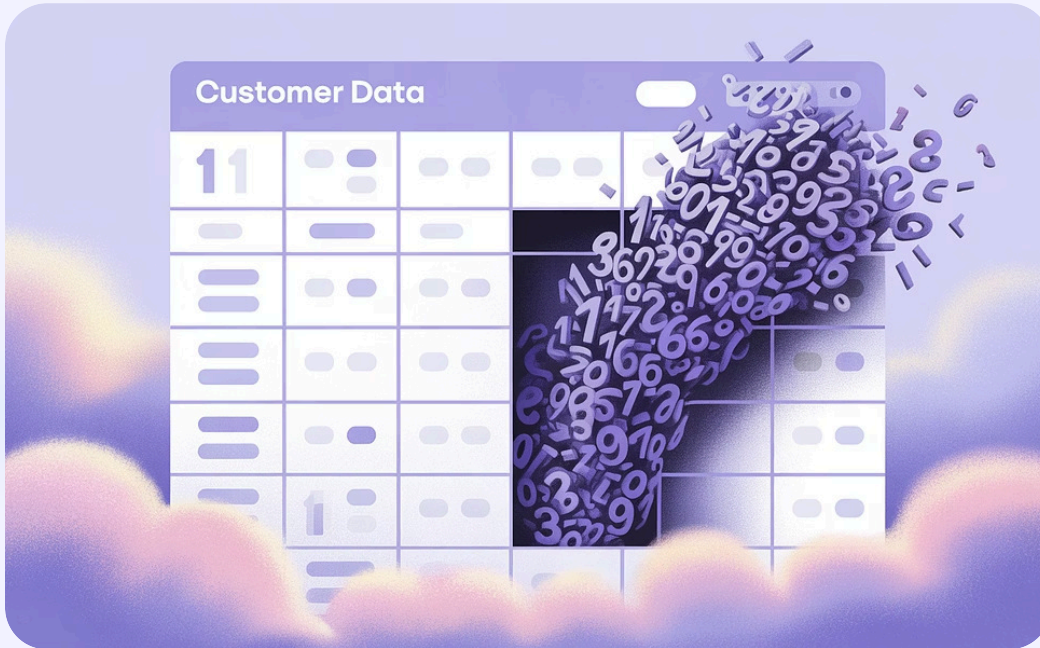
Requisitos para 1FN:

- Cada tabla debe tener una clave primaria
- Cada columna debe contener valores atómicos (indivisibles)
- No repetir grupos de columnas
- Cada registro debe ser único



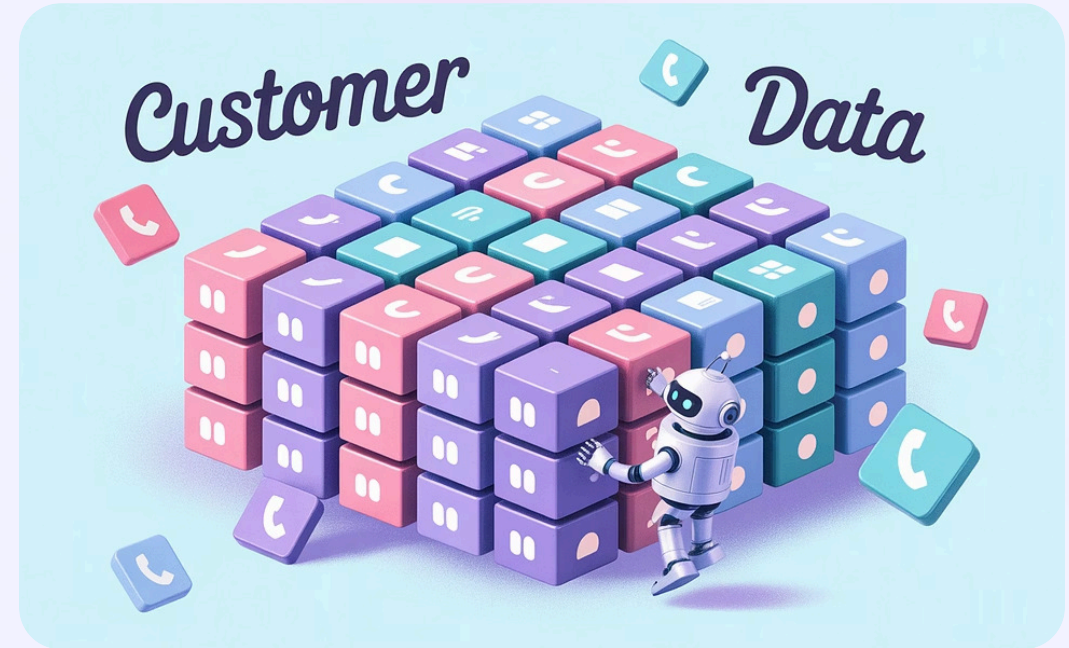
Ejemplo de violación: Una columna "Teléfonos" que contiene varios números de teléfono en un solo campo infringe la 1FN porque los valores no son atómicos.

Ejemplo de 1FN



Antes de 1FN (Infracción)

Tabla de clientes con varios números de teléfono en un solo campo y componentes de dirección repetidos.



Después de 1FN (Cumple)

Datos reestructurados con un teléfono por registro en una tabla separada, valores atómicos para todos los campos.

La conversión a 1FN a menudo requiere la creación de tablas adicionales para manejar correctamente las relaciones de uno a muchos.

Segunda forma normal (2FN)

Requisitos para 2FN:

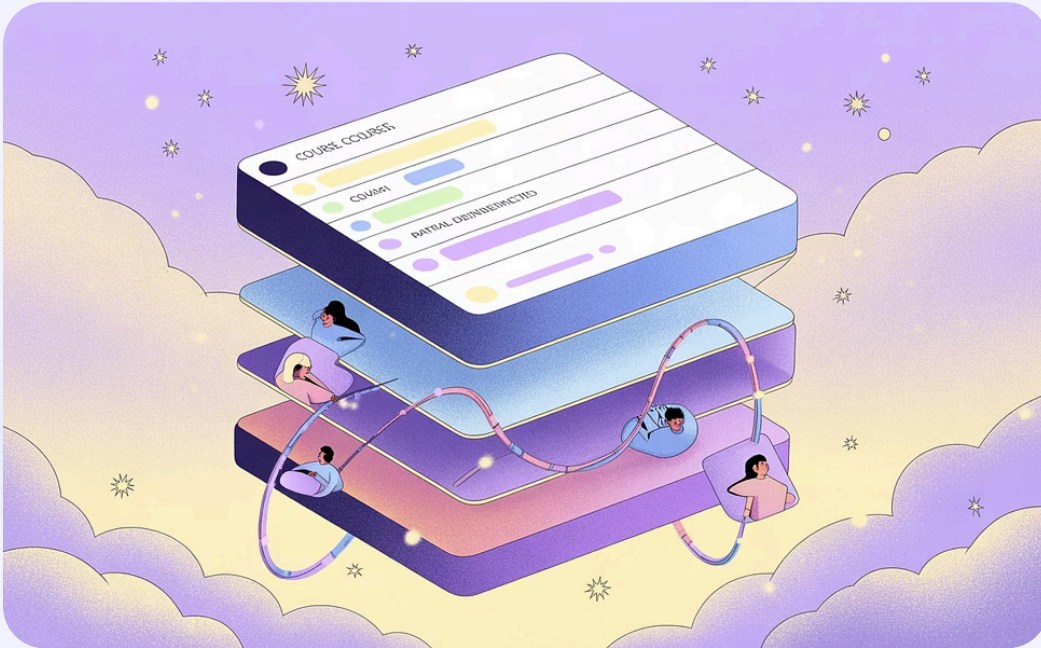
- Ya debe satisfacer la 1FN
- Todos los atributos que no son clave deben depender de la clave principal **completa**
- Particularmente importante cuando se trata de claves compuestas

La 2FN elimina las **dependencias parciales** donde un atributo depende solo de una parte de una clave principal compuesta.



Ejemplo de infracción: En una tabla Estudiante_Curso con clave compuesta (IDEstudiante, IDCurso), si NombreEstudiante depende solo de IDEstudiante, crea una dependencia parcial que rompe la 2FN.

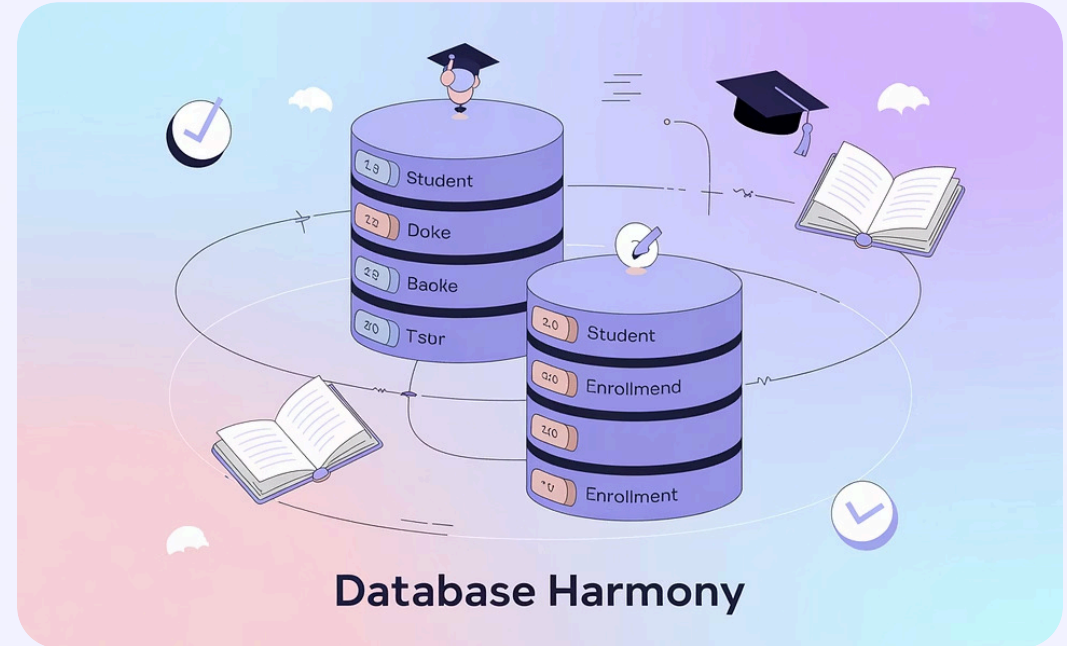
Ejemplo de 2FN



Antes de la 2FN (Infracción)

Tabla StudentCourse donde StudentName depende solo de StudentID, no de la clave compuesta completa (StudentID, CourseID).

El principio clave en 2FN es aislar los datos que dependen solo de una parte de la clave en tablas separadas.



Después de la 2FN (Cumplimiento)

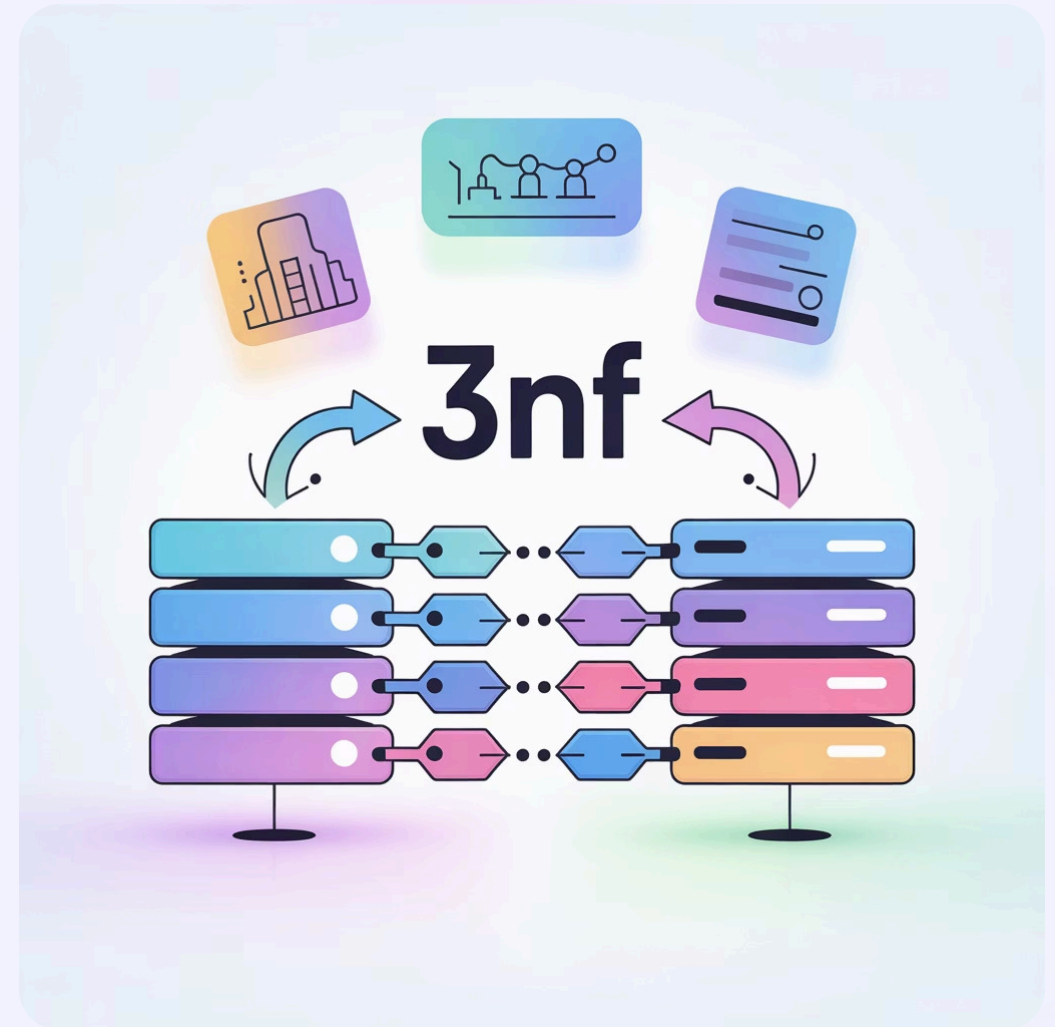
Los atributos del estudiante se movieron a una tabla Student separada, con la tabla Enrollment manteniendo solo los atributos totalmente dependientes.

Tercera forma normal (3FN)

Requisitos para 3FN:

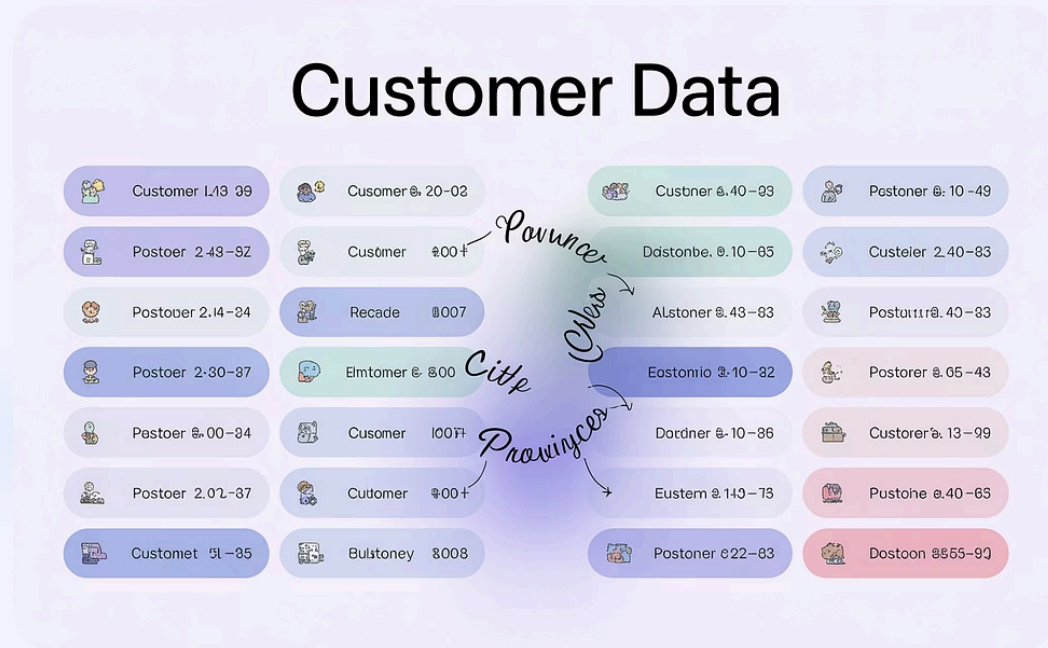
- Ya debe satisfacer la 2FN
- Sin dependencias transitivas entre atributos que no son clave
- Cada atributo que no es clave debe depender directamente de la clave principal

La 3FN elimina las **dependencias transitivas** donde un atributo que no es clave determina otro atributo que no es clave.



Ejemplo de infracción: En una tabla de clientes, si el código postal determina la ciudad y la provincia, estos atributos de ubicación exhiben una dependencia transitiva que rompe la 3FN.

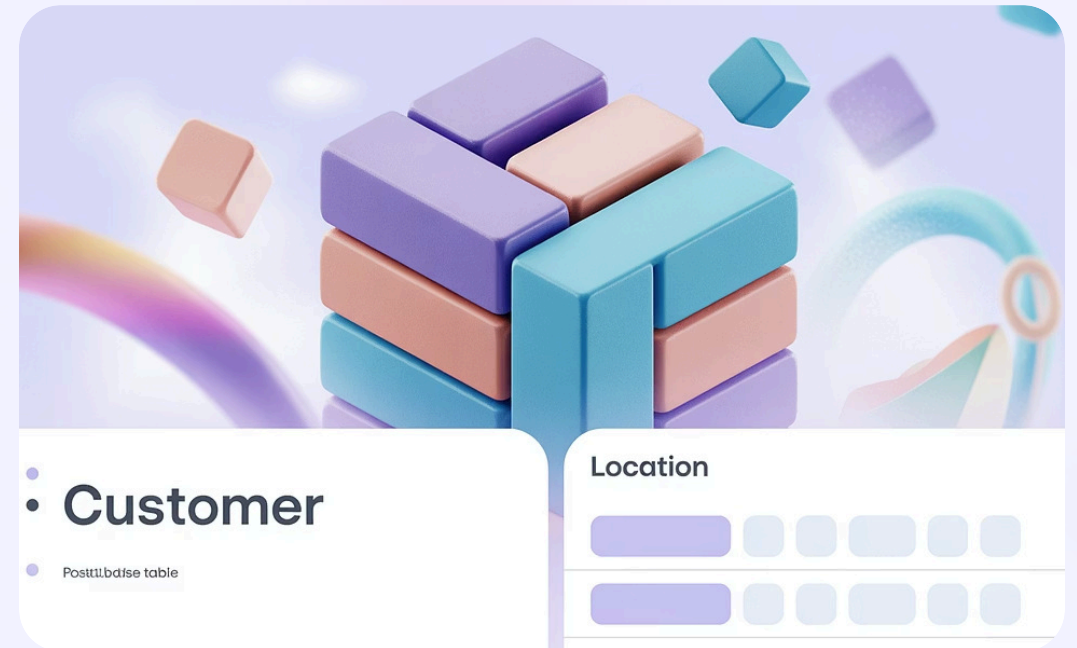
Ejemplo de 3FN



Antes de 3FN (Infracción)

Tabla de clientes donde el código postal determina la ciudad y la provincia, creando una dependencia transitiva no relacionada directamente con la clave principal.

Al aplicar 3FN, nos aseguramos de que todos los atributos dependan solo de la clave, de la clave completa y de nada más que de la clave.



Después de 3FN (Cumple)

Datos de ubicación movidos a una tabla de código postal separada, con la tabla de clientes haciendo referencia a ella a través de una clave externa.

Beneficios de la normalización



Integridad de los datos

Las bases de datos normalizadas previenen inconsistencias y contradicciones en los datos almacenados, manteniendo la precisión en todo el sistema.



Eficiencia de actualización

Los cambios en los datos deben realizarse en un solo lugar, lo que elimina el riesgo de actualizaciones parciales e inconsistencias.



Optimización del almacenamiento

Al eliminar la redundancia, las bases de datos normalizadas requieren menos espacio de almacenamiento y, a menudo, funcionan mejor para las operaciones de escritura.



Flexibilidad de consulta

Las estructuras bien normalizadas permiten realizar consultas e informes más flexibles entre entidades relacionadas.

Posibles desventajas de la normalización

Consideraciones de rendimiento

Si bien la normalización mejora la integridad de los datos, a veces puede afectar el rendimiento de las consultas:

- Más tablas significan más uniones en consultas complejas
- Las operaciones de unión pueden ser computacionalmente costosas
- Algunas aplicaciones con muchas lecturas pueden beneficiarse de la desnormalización estratégica



Cuándo considerar la desnormalización

- Aplicaciones con muchas lecturas y actualizaciones mínimas
- Sistemas de informes donde la velocidad de las consultas es fundamental
- Aplicaciones de almacenamiento de datos

La clave es encontrar el equilibrio adecuado para las necesidades específicas de su aplicación.

Caso práctico de normalización

Tabla no normalizada: Pedidos de clientes

ID del pedido	Nombre del cliente	Correo electrónico del cliente	Teléfono del cliente	ID del producto	Nombre del producto	Categoría	Precio	Cantidad
1001	Maria Garcia	maria@example.com	555-1234	P100	Laptop	Electrónica	\$899	1
1001	Maria Garcia	maria@example.com	555-1234	P200	Mouse	Accesorios	\$25	2
1002	John Smith	john@example.com	555-5678	P100	Laptop	Electrónica	\$899	1

Esta única tabla contiene múltiples anomalías y redundancias. Normalicémosla en la siguiente diapositiva.

Caso de estudio: Solución normalizada

1

Tabla de clientes

CustomerID (PK), CustomerName, CustomerEmail, CustomerPhone

2

Tabla de pedidos

OrderID (PK), CustomerID (FK), OrderDate

3

Tabla de productos

ProductID (PK), ProductName, Category, Price

4

Tabla de detalles de pedidos

OrderID (FK), ProductID (FK), Quantity

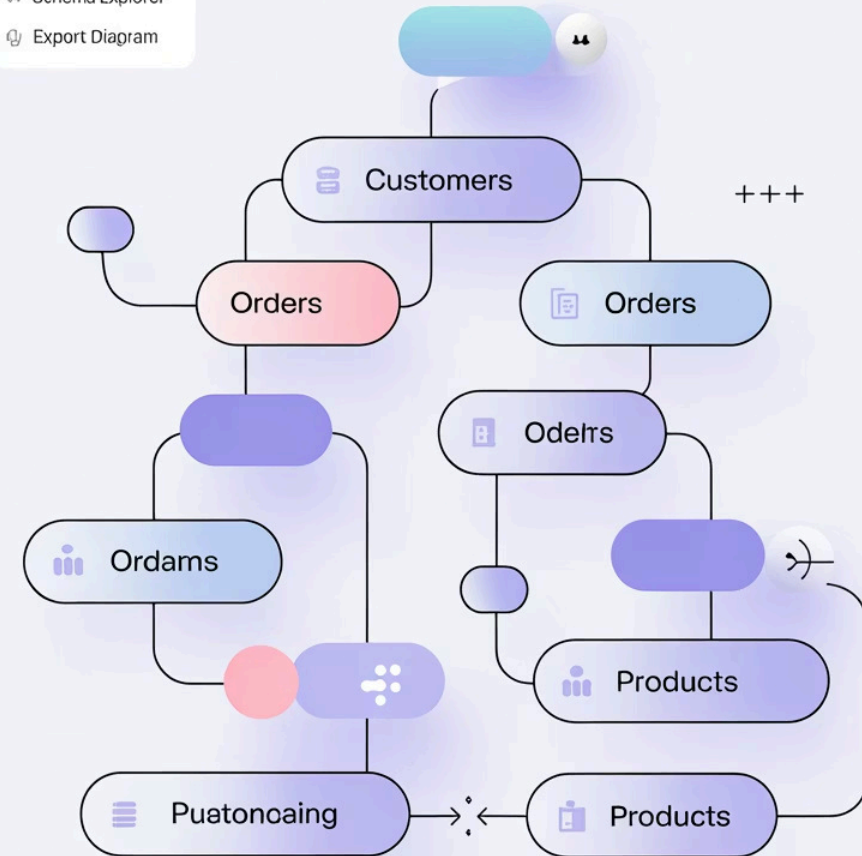
El diseño normalizado elimina la redundancia al tiempo que mantiene todas las relaciones entre las entidades.

Data Flow

Data Dictionary

Schema Explorer

Export Diagram



Ejercicio práctico

Tabla de inicio: No normalizada

ID	Nombre	Teléfono1	Teléfono2	Ciudad	Provincia
1	Ana Lopez	555-1111	555-2222	Barcelona	Cataluña
2	Carlos Ruiz	555-3333	NULL	Madrid	Madrid
3	Elena Vega	555-4444	555-5555	Valencia	Valencia

Su tarea: Aplique 1NF, 2NF y 3NF para normalizar esta estructura de tabla. Identifique los problemas y cree un esquema relacional adecuado.

Problemas a abordar:

- Múltiples números de teléfono en columnas separadas (violación de 1NF)
- La ciudad determina la provincia (dependencia transitiva para 3NF)

Considere cómo estructuraría las relaciones entre las tablas normalizadas.



Conclusiones clave

El proceso de diseño importa

Siga el enfoque de tres fases (conceptual, lógico, físico) para un diseño de base de datos integral.

Beneficios de la normalización

La aplicación de formas normales elimina la redundancia, mejora la integridad de los datos y crea estructuras de bases de datos fáciles de mantener.

El equilibrio es clave

Considere las compensaciones de rendimiento al normalizar, particularmente para aplicaciones con muchas lecturas donde cierta desnormalización podría ser beneficiosa.

Dominar los principios de diseño y normalización de bases de datos lo ayudará a crear sistemas de bases de datos robustos, eficientes y fáciles de mantener a lo largo de su carrera.

